



COURSE SLIDES

# Agentic AI for Video Creation

---

Course Code: C436

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v2.0 · 16 September 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · [www.tertiarycourses.com.sg](http://www.tertiarycourses.com.sg)



COURSE ADMINISTRATION

# Welcome & Housekeeping

# Course Trainer Profiles

1

## Surendra Batukdeo

Emerging-technology, automation, data-intelligence, project, and technical-training experience.

2

## Agus Salim

IT solutions, systems integration, cloud infrastructure, project, and cybersecurity experience.

3

## Quah Chee Yong

Data science, Python, natural-language processing, machine-learning, and AI-training experience.

# Let's Know Each Other

1 Introduce yourself and name one short-form video workflow you would like to improve.

2 Identify the stage that consumes the most time or creates the most rework.

3 State one video action you would never allow an agent to take without your approval.

# Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

One conversation at a time.

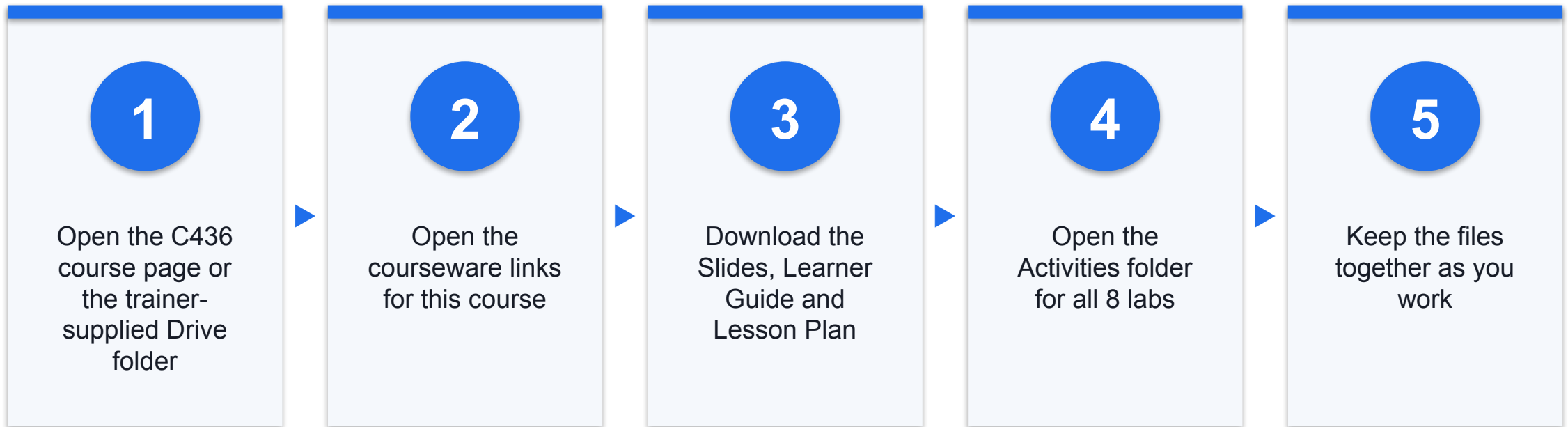
5

Be punctual; return from breaks on time.

6

Learn by doing — try every hands-on lab.

# Access Course Materials



Course page: <https://www.tertiarycourses.com.sg/agent-ai-for-video-creation.html>

# Lesson Plan — 2 Days, 15 instructional hours

### Course sequence

- Day 1 — Configure Hermes, engineer video prompts and route the media tools
  - Topic 1: Hermes Agent Setup, MiniMax M3 and Video Prompt Engineering (Labs 1–3)
  - Topic 2: Video Tools, Hermes Skills and Custom Brand Production (Labs 4–5)

### Day 2

- Day 2 — Build the branded video, then orchestrate review and controlled release
  - Topic 3: Multi-Agent Kanban, YouTube Release and Scheduled Publishing (Labs 6–8)
- Daily timing
  - 9:30 am - 6:30 pm (1-hour lunch; two 15-minute tea breaks; 7.5 instructional hours)

# Learning Outcomes

1

## Plan the Video

Develop an editing strategy and work plan in Hermes Agent using MiniMax M3, structured video prompts and governed tool selection.

2

## Build the Video

Create and customise an evidence-backed video with Hermes skills, Remotion, Manim, Higgsfield-compatible requests and deterministic media checks.

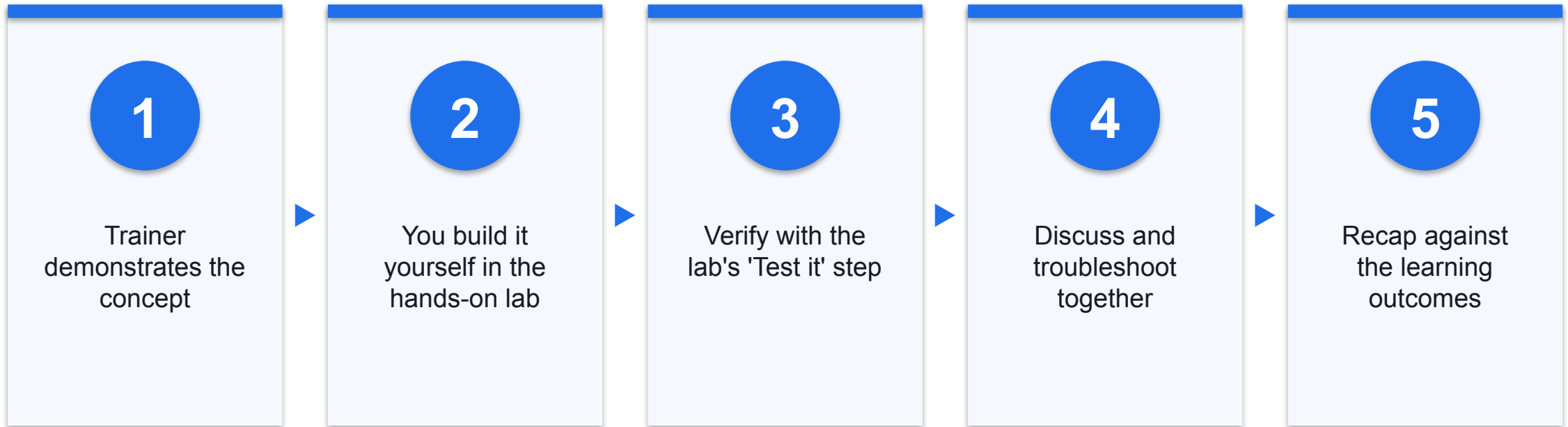
3

## Orchestrate and Release

Orchestrate research, production, review and YouTube release agents through a durable Kanban board and controlled scheduled publishing.

# How You'll Learn

---





CORE CONCEPTS

# Agentic Video Production Foundations

# What Makes a Video Workflow Agentic

1

## Goal and state

The system knows the approved brief, the accepted artifacts, the current stage, and the unresolved work.

2

## Reasoning and routing

MiniMax M3 chooses among bounded next actions while deterministic rules enforce known constraints.

3

## Tools and skills

Every capability has typed inputs, portable outputs, permissions, budgets, and failure behaviour.

4

## Evidence and control

Ledgers, hashes, probes, approvals, and stop rules keep the work reviewable.

# The C436 Agent Loop

1

## 1 | Observe

Read the approved brief, the current state, and the asset manifest.

2

## 2 | Plan

Produce a validated shot plan that totals the approved duration.

3

## 3 | Act

Call one allowed tool or skill with structured inputs and a capped attempt count.

4

## 4 | Inspect

Probe the artifact and check it against the declared delivery specification.

5

## 5 | Decide

Accept, repair within limits, or stop and record the stop reason.

6

## 6 | Record

Persist the ledger entry, the hash, and the approval decision before continuing.

## CORE PRINCIPLE

# Autonomy is a design variable, not the goal.

Use deterministic tools for known operations, model judgment for bounded choices, and human approval for consequential actions.

# The Connected Hermes Build

## Configure

- Hermes runtime
- MiniMax M3
- Credential isolation

## Prompt

- FRAME-CUT contract
- Shot plan JSON
- Preview render

## Produce

- Tool routing
- Brand tokens
- Custom video skill

# How Every Lab Progresses

1

Open the lab folder as the current Hermes project and read its copy-ready prompts.

2

Ask Hermes to inspect the supplied local files before it proposes a plan.

3

Require a preview before any network, paid-generation, upload, or scheduling side effect.

4

Run the lab's `verify.py` and retain the PASS output with the requested evidence.

5

Reflect on which choices should remain deterministic, model-led, or human-approved.

# Autonomy Matrix

1

## Deterministic

Schema validation, duration arithmetic, media probing, hashing, and file naming.

2

## Model-assisted

Shot planning, prompt repair, review findings, and research clustering.

3

## Human-approved

Claims, rights, brand exceptions, the final master, and every release action.

4

## Prohibited by default

Unreviewed public posts, credential exposure, and living-artist imitation.

# The FRAME-CUT Contract

1

## Format and finish

Channel, duration, aspect ratio, and delivery codec frozen before shot work.

2

## Role and references

Assigned role, attached brief, registered sources, and authorised assets.

3

## Action and motion

One dominant subject action and one camera move per timed shot.

4

## Continuity and unwanted

Identity and palette tokens to keep, plus explicit must-avoid elements.

# Release Evidence

1

## Identity

Run ID, plan hash, master hash, asset version, and idempotency key.

2

## Actions

Tool or skill name, input reference, output reference, status, and error.

3

## Controls

Probe result, finding ID, approval hash, reviewer, and decision.

4

## Resources

Attempts, latency, estimated cost, and budget variance.

TOPIC 01

# Hermes Agent Setup, MiniMax M3 and Video Prompt Engineering

Hermes Desktop runtime | MiniMax M3 provider contract | FRAME-CUT prompting | Structured shot plans

## Key Concepts — Hermes Agent Setup, MiniMax M3 and Video Prompt Engineering

1

### Runtime boundary

The installed Hermes surface, its profile, configuration path and skill root that every run depends on.

2

### Provider contract

The named provider, base URL, model identifier and credential reference used for reasoning.

3

### FRAME-CUT

A prompt contract covering Format, Role, Action, Motion, Environment plus Continuity, Unwanted and Technical output.

4

### Shot plan

Strict JSON of timed shots whose durations total the approved target with no gaps or overlaps.

# Hermes Desktop Runtime and Shared CLI State

## WHAT

- Hermes Agent runs as a desktop application with a matching command-line surface. Both read the same profile, configuration path and skill directory, so a change made in one surface is visible to the other. Installation is completed from the official desktop page, after which a diagnostic command reports the health of each required dependency. The runtime boundary is the set of facts that define where the agent executes: platform, installer source, version, active profile, configuration path and skill root.

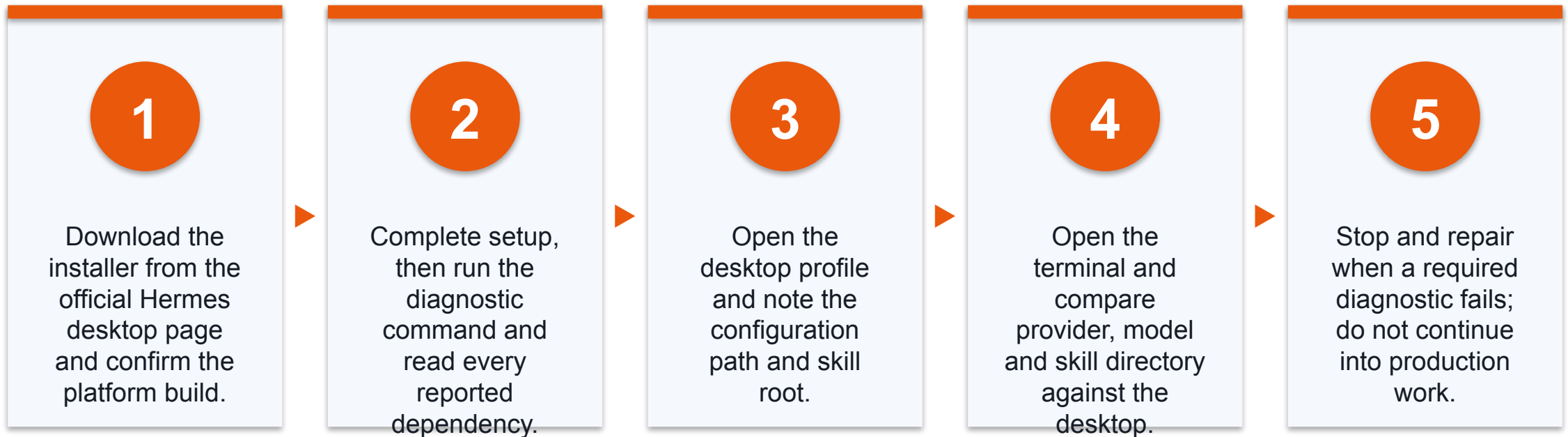
## WHY IT MATTERS

- An agent that cannot describe its own runtime cannot produce reproducible work. If the desktop and the terminal disagree about the active provider or skill directory, a lab that passes in one surface fails in the other for reasons that look like model behaviour. Recording the runtime boundary before any production work converts a class of confusing failures into a single observable check, and a failed dependency becomes a reason to stop rather than a problem to discover halfway through a render.

## HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — Hermes Desktop Runtime and Shared CLI State



## In Practice — Hermes Desktop Runtime and Shared CLI State

### WORKED EXAMPLE

- hermes doctor reports all required dependencies healthy on a supported platform.
- The desktop profile and the CLI session report the same provider, model and skill root.
- setup-evidence.json records the official source, version and diagnostic status with no credential value.

### DECISION GUIDE

- **Use when**
  - A new machine, a new learner account or a fresh profile is being prepared for production work.
  - Results differ between the desktop and the terminal and the cause is not yet known.
- **Avoid when**
  - A required diagnostic fails and the failure has not been repaired or recorded.
  - The installer came from an unofficial mirror or the platform build is unverified.

## Quality Lens — Hermes Desktop Runtime and Shared CLI State

### Failure signal

- The same prompt behaves differently in the desktop and the terminal.

### Repair move

- Name the active profile and compare provider, model and skill directory across both surfaces.

### Quality evidence

- profile-check.json shows matching settings and the diagnostic reports no blocking failure.

## MiniMax M3 Provider, Trial Terms and Credential Isolation

### WHAT

- The provider contract names the service, base URL, model identifier and a reference to a stored credential. This course configures provider minimax with model MiniMax-M3 and verifies a tool-capable response before any production run. Trial availability, duration, quota and region terms are time-sensitive commercial terms rather than fixed course facts, so they are observed live at sign-up and recorded with a date. Credentials live in Hermes credential or configuration storage and are referenced by name; published evidence carries placeholders only.

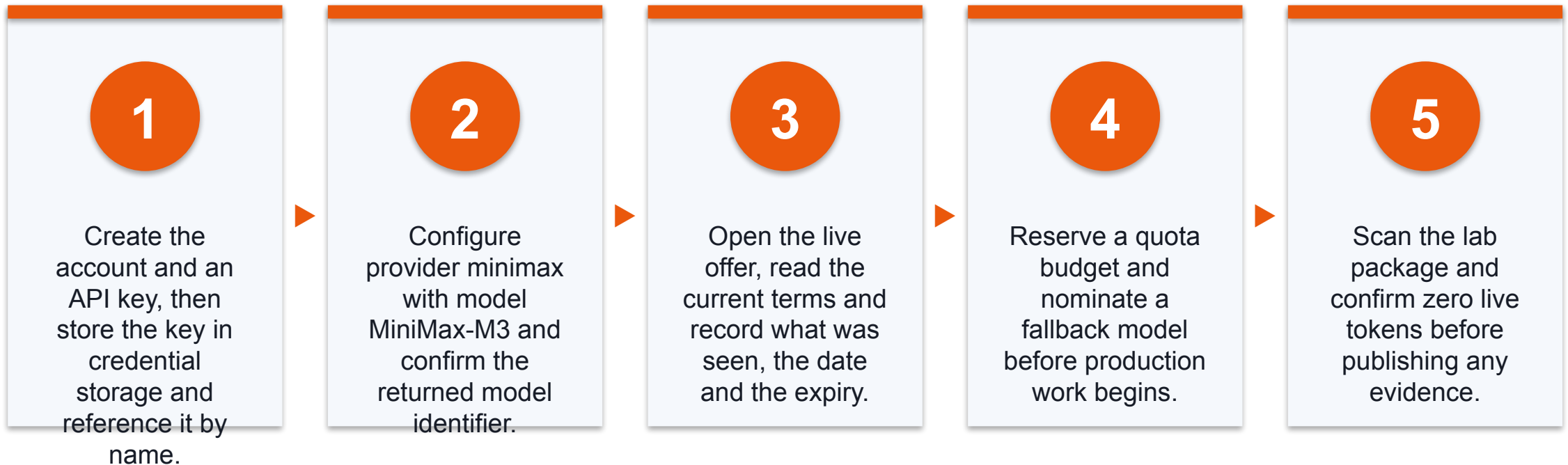
### WHY IT MATTERS

- A model handshake that is assumed rather than verified is the most common cause of a lab that fails much later, at the point of an expensive call. Verifying the returned model identifier proves that the configured model is the model actually answering. Separating the credential from the evidence keeps the lab package publishable: a secret that never enters a prompt, file, screenshot or repository cannot be leaked by one of them. Recording trial terms with a date and a fallback model means a changed offer becomes a planned decision rather than a blocked class.

### HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — MiniMax M3 Provider, Trial Terms and Credential Isolation



## In Practice — MiniMax M3 Provider, Trial Terms and Credential Isolation

### WORKED EXAMPLE

- model-check.json records MiniMax-M3 and a successful tool-capable response without recording the key.
- trial-checklist.md contains the observed terms, the date seen and the learner's fallback decision.
- A secret scan over the lab package reports no live token pattern.

### DECISION GUIDE

- **Use when**

- A provider or model is being configured for the first time, or a key has been rotated.
- Quota, billing or region terms materially affect how much generation the run can afford.

- **Avoid when**

- The returned model identifier does not match the configured model.
- A key would have to be pasted into a prompt, lab file, screenshot or repository to proceed.

## Quality Lens — MiniMax M3 Provider, Trial Terms and Credential Isolation

### Failure signal

- The session answers but reports a different model than the one configured.

### Repair move

- Start a new session after selecting the model; an existing session can retain its original model.

### Quality evidence

- budget-ledger.csv reconciles attempts, latency and cost against the approved ceiling.

# The FRAME-CUT Video Prompt Contract

## WHAT

- FRAME-CUT is the prompt contract used throughout this course. FRAME fixes Format and finish, Role and references, Action, Motion and Environment. CUT fixes Continuity tokens, Unwanted elements as explicit negative constraints, and the Technical output schema. A compliant prompt names one dominant subject action and one camera move per timed shot, states the palette and identity tokens that must persist, lists what must never appear, and requires strict JSON that can be validated before anything is generated.

## WHY IT MATTERS

- A vague creative request such as 'make it cinematic' leaves the model to invent the channel, duration, subject, camera language and acceptance rule, and every regeneration invents them differently. A field-complete contract makes shots comparable across attempts, makes drift observable against named invariants, and makes the output checkable by a validator rather than by opinion. Because the contract is validated before generation, a defective plan costs a schema error instead of a paid render.

## HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — The FRAME-CUT Video Prompt Contract



## In Practice — The FRAME-CUT Video Prompt Contract

### WORKED EXAMPLE

- A 15-second brief becomes a shot plan whose shot durations total exactly 15 seconds with no overlap.
- Each shot carries one action-motion pair, a continuity token and an explicit must-avoid list.
- prompt-contract.json passes the schema and duration checks before a renderer is called.

### DECISION GUIDE

- **Use when**
  - A creative intent must survive several regenerations or be handed to another agent or tool.
  - The output will feed a deterministic renderer that requires named, typed fields.
- **Avoid when**
  - The request asks for imitation of a living artist or a named creator's identity.
  - The delivery format is still unsettled, so shot work would be rebuilt after the format changes.

## Quality Lens — The FRAME-CUT Video Prompt Contract

### Failure signal

- Shots look plausible individually but identity, palette or screen direction drifts between them.

### Repair move

- Reuse the continuity token and compare keyframes against the explicit invariant list.

### Quality evidence

- continuity-review.json records each deviation and the acceptance decision.

# The Simple Video Agent Loop with Cost and Stop Rules

## WHAT

- The agent loop receives a brief, plans shots, calls one tool, inspects the returned artifact and then stops or escalates. Its controls are numeric: an estimated cost, a maximum number of attempts, a timeout and a recorded stop reason. A run ledger links the run identifier, the plan hash, each tool call and each result so that the run can be reconstructed after the fact. The default posture is one render attempt, one review pass and an explicit repair decision rather than open-ended retrying.

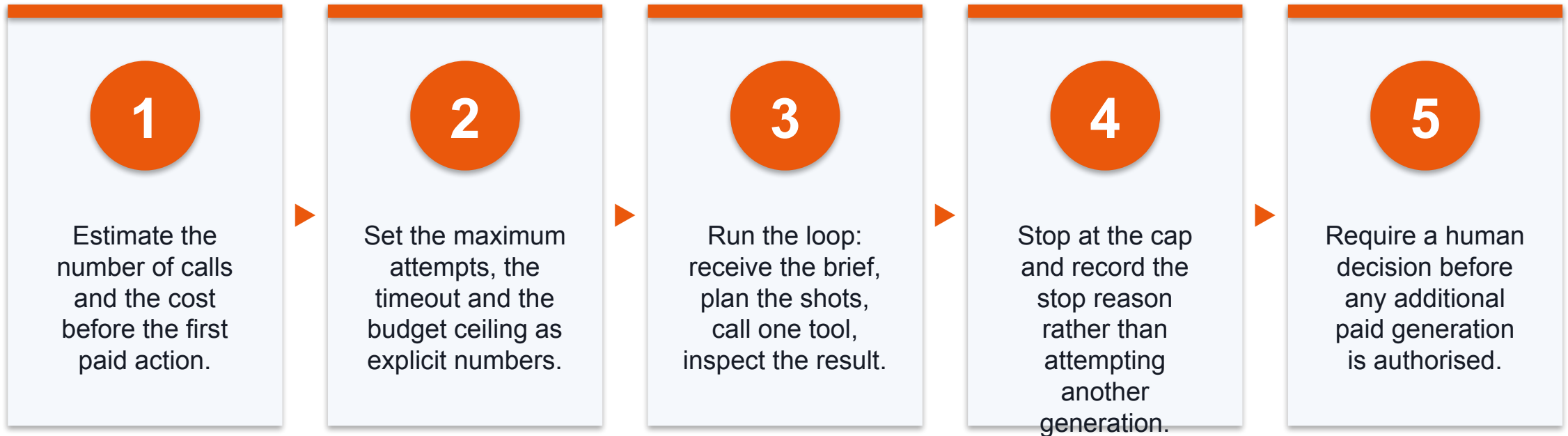
## WHY IT MATTERS

- Generation costs money and time, and an unbounded loop spends both without converging. Capping attempts converts a runaway loop into a decision that reaches a named person with the evidence already assembled. Recording the stop reason matters as much as recording success: a run that stopped because a dependency failed is a different operational fact from one that stopped because the budget ceiling was reached, and the two require different repairs.

## HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — The Simple Video Agent Loop with Cost and Stop Rules



## In Practice — The Simple Video Agent Loop with Cost and Stop Rules

### WORKED EXAMPLE

- run-ledger.json contains the plan, the tool call, the result reference and the stop reason.
- A 15-second preview is produced in one render attempt and probed before it is accepted.
- A second paid attempt is not started automatically; it is escalated with the evidence attached.

### DECISION GUIDE

- **Use when**
  - Work involves paid generation, external calls or any action with a real cost.
  - A run must be explainable later to a reviewer who was not present when it executed.
- **Avoid when**
  - Attempts, timeouts and budget have not been given numeric limits.
  - The loop can start another paid generation without a recorded human decision.

## Quality Lens — The Simple Video Agent Loop with Cost and Stop Rules

### Failure signal

- The loop keeps regenerating without reaching a defined finish state.

### Repair move

- Cap attempts, add an explicit finish condition and route the remainder to a named owner.

### Quality evidence

- The ledger shows one committed result per run identifier and a recorded stop reason.

## Hands-On Labs — Hermes Agent Setup, MiniMax M3 and Video Prompt Engineering

### Lab 1

- Set Up Hermes Desktop and Connect MiniMax M3

### Lab 2

- Prompt Hermes to Create a Simple Video

### Lab 3

- Engineer Video Prompts with FRAME-CUT

# Set Up Hermes Desktop and Connect MiniMax M3

## LAB 1

You will install Hermes Desktop from the official source, configure the MiniMax provider for MiniMax-M3, run the diagnostic command, compare the desktop and CLI state, and record the live trial terms you actually see. Every piece of published evidence carries placeholders instead of secrets.

### You'll build

**A setup-evidence.json recording the official source, version and diagnostic status, plus a redacted diagnostic screenshot.**

1 Download the official installer

2 Complete Hermes setup

3 Configure MiniMax-M3

4 Run hermes doctor

**Tools:** Hermes Desktop, Hermes CLI, MiniMax M3, credential storage

## Set Up Hermes Desktop and Connect MiniMax M3

---

### Test it

verify.py must print PASS Lab 01. Hermes launches, the configured provider is minimax, the model is MiniMax-M3, hermes doctor reports no blocking failure, and the published evidence contains placeholders only.

# Prompt Hermes to Create a Simple Video

## LAB 2

You will use a copy-ready Hermes prompt to turn a supplied 15-second brief into a strict-JSON shot plan, validate that the plan totals the target duration with no gaps or overlaps, run the deterministic preview renderer and probe the resulting MP4.

### You'll build

**A simple-video.mp4 with its shot-plan.json and ffprobe.json evidence.**

1 Open the project folder

2 Submit the bounded prompt

3 Validate the shot plan

4 Run the preview renderer

**Tools:** Hermes Desktop, MiniMax M3, Python preview renderer, FFprobe

# Prompt Hermes to Create a Simple Video

---

## Test it

verify.py must print PASS Lab 02. The shot plan is valid JSON, totals 15 seconds, uses only supplied assets, and the generated MP4 passes the dimensions, codec and duration checks.

# Engineer Video Prompts with FRAME-CUT

## LAB 3

You will diagnose a deliberately vague prompt, complete every FRAME field and every CUT field, generate strict shot JSON, score the result against a deterministic rubric and repair the single highest-severity defect.

### You'll build

**A prompt-pack.json with its prompt-score.csv rubric evidence.**

1 Diagnose the vague prompt

2 Complete the FRAME fields

3 Complete the CUT fields

4 Generate the shot JSON

**Tools:** Hermes Desktop, MiniMax M3, FRAME-CUT template, scoring rubric

# Engineer Video Prompts with FRAME-CUT

---

## Test it

verify.py must print PASS Lab 03. Every shot carries all FRAME-CUT fields, one dominant action, one camera move, explicit continuity and negative constraints, and a measurable output check.

# Recap — Hermes Agent Setup, MiniMax M3 and Video Prompt Engineering

1

You can now: LO1: configure the Hermes runtime and prove a tool-capable MiniMax M3 handshake without exposing a credential

2

You can now: LO1: convert a bounded brief into a validated shot plan and a deterministically rendered preview

3

You can now: LO1: apply the FRAME-CUT contract to make creative intent portable, comparable and checkable

TOPIC 02

# Video Tools, Hermes Skills and Custom Brand Production

Tool versus skill | Remotion, Manim and FFmpeg | Brand tokens | Technical quality gate

## Key Concepts — Video Tools, Hermes Skills and Custom Brand Production

1

### Tool versus skill

A tool performs precise integrated execution; a skill packages repeatable instructions plus scripts.

2

### Skill anatomy

A directory holding SKILL.md with frontmatter, a procedure, bundled scripts and verification commands.

3

### Brand token

A stored, editable value for logo, palette, typography or motion referenced by the video skill.

4

### Quality gate

A fail-closed technical check on container, streams, dimensions, frame rate, duration and captions.

# Choosing Between a Tool and a Hermes Skill

## WHAT

- A tool is the right choice when a capability needs precise, integrated execution with its own authentication and error handling. A skill is the right choice when a procedure must be repeated reliably: it bundles instructions, scripts and templates in a directory containing SKILL.md, whose frontmatter declares the name, description and version. Hermes discovers skills by searching the catalogue, then loads the selected skill and only the resources it references, so context stays proportionate to the task.

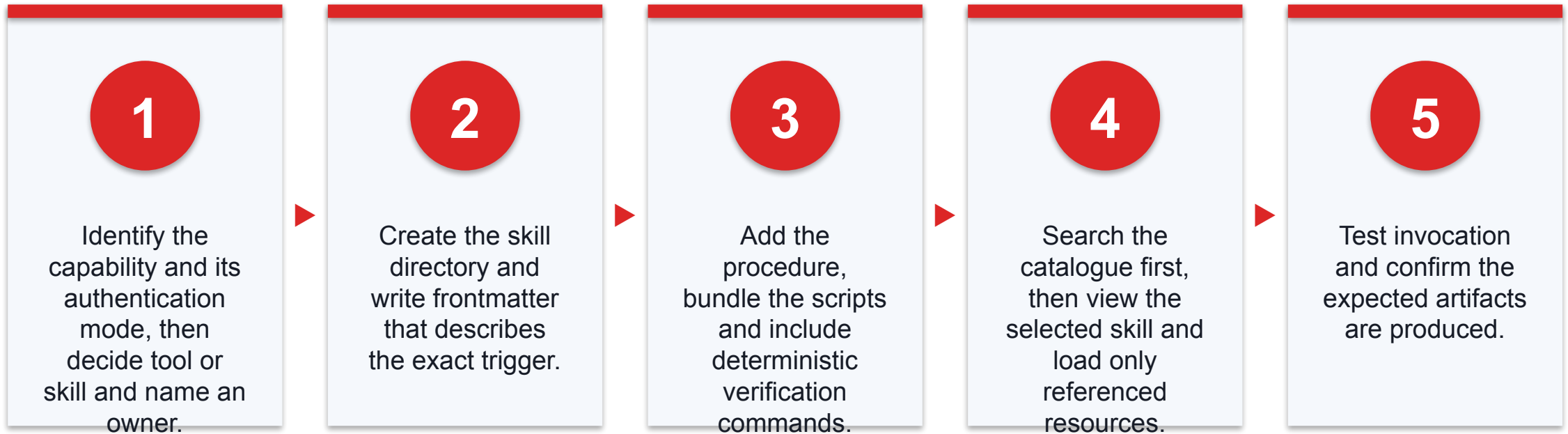
## WHY IT MATTERS

- Wrapping everything as a skill produces brittle prose where an integration was needed; wrapping everything as a tool produces unrepeatable one-off calls where a documented procedure was needed. Deciding explicitly, and recording why, keeps the production stack legible to the next person. Progressive loading matters for the same reason: a skill that pulls its entire reference tree into context on every invocation crowds out the working material the task actually needs.

## HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — Choosing Between a Tool and a Hermes Skill



## In Practice — Choosing Between a Tool and a Hermes Skill

### WORKED EXAMPLE

- decision-record.md states why each capability was implemented as a tool or as a skill.
- SKILL.md and a test transcript together show correct activation from the intended trigger.
- skill-usage.json records the selected skill and the resources that were loaded.

### DECISION GUIDE

- **Use when**
  - A procedure will be repeated across runs, learners or courses and must behave the same way each time.
  - The capability needs bundled scripts and templates alongside its instructions.
- **Avoid when**
  - A single ad-hoc call would do and no repetition is expected.
  - The capability requires integrated authentication and precise error handling better served by a tool.

## Quality Lens — Choosing Between a Tool and a Hermes Skill

### Failure signal

- A skill is not discovered when its trigger phrase is used.

### Repair move

- Check the YAML frontmatter, the directory name and the SKILL.md filename, then restart discovery.

### Quality evidence

- Repeated runs of the skill produce matching outputs from a clean start.

# Routing Remotion, Manim, Higgsfield and FFmpeg

## WHAT

- Each media tool has a distinct contract. Remotion renders code-driven compositions where duration in frames divided by frames per second gives the duration in seconds. Manim renders programmatic explanatory scenes whose labels and values are bound to an approved evidence model. A Higgsfield-compatible request selects a model, attaches references and writes a motion prompt, and is prepared as a preview before any account or quota is consumed. FFmpeg performs deterministic assembly: normalise every source, build the timeline, mix audio and encode the master.

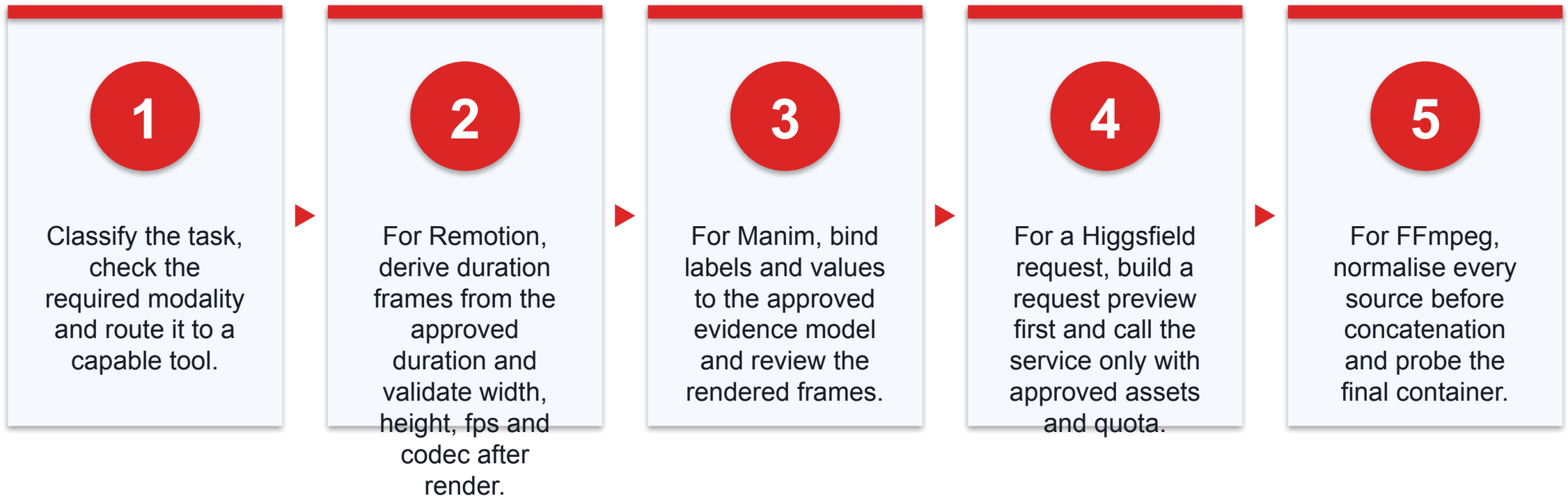
## WHY IT MATTERS

- Routing a task to a tool that cannot satisfy its contract is the most expensive category of error in a video pipeline, because the cost is usually discovered after generation. Deriving frames from duration rather than guessing prevents a composition that is silently the wrong length. Normalising sources before concatenation prevents the class of failures where clips join but the container reports inconsistent streams. Preparing a generation request as a preview keeps an unapproved asset or an exhausted quota from becoming a failed paid call.

## HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — Routing Remotion, Manim, Higgsfield and FFmpeg



## In Practice — Routing Remotion, Manim, Higgsfield and FFmpeg

### WORKED EXAMPLE

- tool-routing.csv links every task to a capable model or tool with a recorded fallback.
- remotion-render.json and ffprobe.json agree with the composition contract.
- ffprobe.json confirms H.264 video, AAC audio, the expected dimensions, frame rate and duration.

### DECISION GUIDE

- **Use when**
  - A media task has a clear modality and an approved tool exists for it.
  - The output must be verified deterministically rather than accepted on appearance.
- **Avoid when**
  - The required assets are unapproved or the account quota has not been confirmed.
  - A deterministic renderer would produce the result more reliably than a generative call.

## Quality Lens — Routing Remotion, Manim, Higgsfield and FFmpeg

### Failure signal

- Clips concatenate but the final container reports inconsistent streams or the wrong duration.

### Repair move

- Normalise every source to the same codec, dimensions and frame rate before rebuilding the timeline.

### Quality evidence

- The probe of the final master matches the approved delivery specification field for field.

# Brand Tokens, Tone and the Custom Video Skill

## WHAT

- A brand token system stores the logo reference, palette values, font stack and motion rules as editable values that the video skill references rather than hard-codes. Tone and style controls describe the audience, the voice, the pacing in words per minute and an explicit list of stylistic exclusions. The custom brand-video skill combines these: it bundles scripts and templates, declares its inputs and outputs, and ships an acceptance test so that repeated runs can be compared rather than merely rerun.

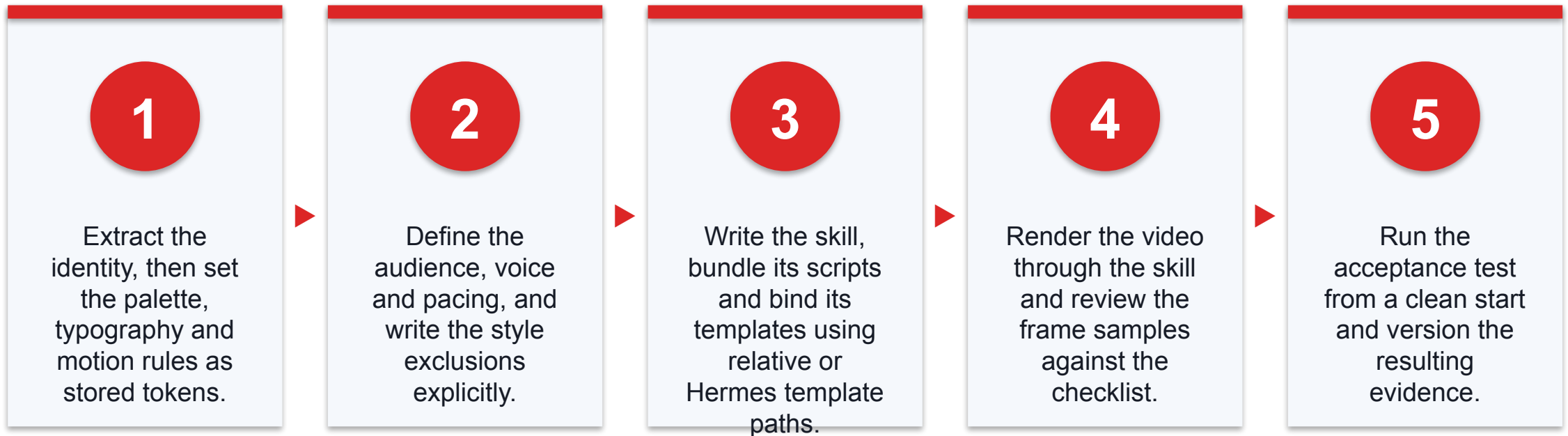
## WHY IT MATTERS

- Brand rules written into prose drift as soon as two people apply them. Stored as tokens and referenced from a skill, they become checkable: a frame either uses the approved palette value or it does not. Describing tone through attributes and exclusions, rather than by naming a living artist to imitate, keeps the output original and defensible. An acceptance test converts 'the skill works' into a repeatable comparison between a clean run and the expected artifacts.

## HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — Brand Tokens, Tone and the Custom Video Skill



## In Practice — Brand Tokens, Tone and the Custom Video Skill

### WORKED EXAMPLE

- brand-profile.yaml holds editable tokens and the frame samples pass the brand checklist.
- The custom skill is discoverable, produces an MP4 and passes the technical probe.
- A repeat run from a clean state produces matching outputs and the same evidence artifacts.

### DECISION GUIDE

- **Use when**

- The same brand treatment must be applied across many videos or by several people.
- Brand compliance needs to be demonstrated rather than asserted.

- **Avoid when**

- The style brief asks for imitation of a living artist or a specific creator's identity.
- Brand values are still changing, so tokens would be rewritten immediately after use.

## Quality Lens — Brand Tokens, Tone and the Custom Video Skill

### Failure signal

- Two runs of the same skill produce visibly different brand treatment.

### Repair move

- Move the varying value into a stored token and reference it from the skill.

### Quality evidence

- brand-review.json records the passed checks against the applicable brand tokens.

# The Technical Quality Gate, Provenance and Repair

## WHAT

- The quality gate probes the container, inspects representative frames, checks that captions are readable and then either passes or fails the master. It fails closed: a missing stream, an out-of-bounds dimension or an unreadable caption blocks the master rather than producing a warning. Provenance runs alongside it — every asset is registered with its source, licence and a SHA-256 hash, so approval binds to an immutable version. A finding opens a repair, the repair renders a new version, and every affected check is rerun against that new version.

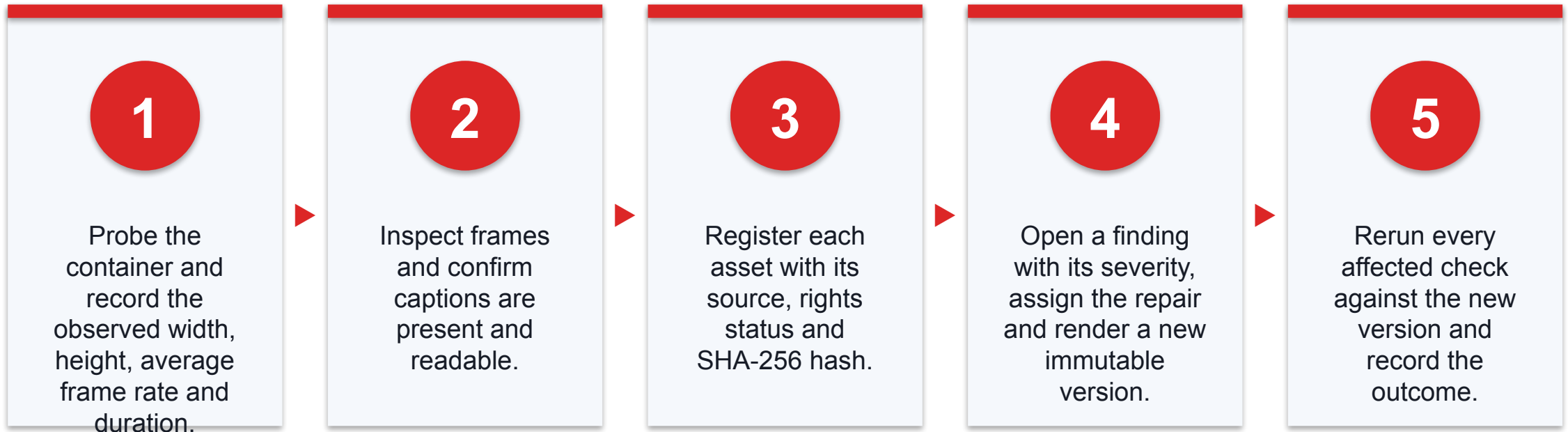
## WHY IT MATTERS

- A video that looks correct in a player can still be technically invalid for the destination platform, and the failure surfaces at upload rather than in review. Deterministic probing catches that class of defect while it is still cheap to fix. Binding approval to a hash prevents the most damaging version-control error in a release pipeline: approving one master and shipping another. Re-testing the affected scope after a repair is what stops a fix in one place from silently breaking another.

## HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — The Technical Quality Gate, Provenance and Repair



## In Practice — The Technical Quality Gate, Provenance and Repair

### WORKED EXAMPLE

- technical-qc.json and the contact sheet show the observed values against the required bounds.
- asset-manifest.csv verifies rights status and hashes for every manifest row.
- repair-log.json links the finding, the change, the new hash and the re-test result.

### DECISION GUIDE

- **Use when**

- A master is a candidate for release and must meet a declared delivery specification.
- Approval will be given now and acted on later, so the approved artifact must be identifiable.

- **Avoid when**

- The delivery specification has not been agreed, so there is nothing to check against.
- A finding would be closed without re-testing the checks it affects.

## Quality Lens — The Technical Quality Gate, Provenance and Repair

### Failure signal

- A repair closes one finding and reopens another that was previously passing.

### Repair move

- Write a new immutable version and rerun the full affected scope rather than the single check.

### Quality evidence

- Closed findings outnumber reopened findings and each closure names its re-test.

## Hands-On Labs — Video Tools, Hermes Skills and Custom Brand Production

### Lab 4

- Install Video Tools and Hermes Skills

### Lab 5

- Create a Custom Branded Video Skill

# Install Video Tools and Hermes Skills

## LAB 4

You will inventory the local media tools, search the Hermes skill catalogue, install or create skills for Remotion, Manim, Higgsfield request preparation and FFmpeg verification, run a smoke test on each, and record the authentication mode, side effects and fallback for every entry in the registry.

### You'll build

**A tool-routing.json registry with skill-smoke-test.json evidence.**

1 Inventory the local tools

2 Search the Hermes skills

3 Install or create the skills

4 Run the smoke tests

**Tools:** Hermes skills, Remotion, Manim, Higgsfield request preview, FFmpeg

# Install Video Tools and Hermes Skills

---

## Test it

verify.py must print PASS Lab 04. The registry records capability, installation state, auth mode, side effects and fallback, and every supplied SKILL.md passes its frontmatter and path checks.

# Create a Custom Branded Video Skill

## LAB 5

You will approve a brand profile, write SKILL.md for a custom brand-video skill, bind its templates using relative or Hermes template paths, render a custom video through the skill, review the frame samples against the brand checklist and version the resulting evidence.

### You'll build

**A custom-video.mp4 with brand-review.json and render-evidence.json.**

1 Approve the brand profile

2 Create SKILL.md

3 Bind the templates

4 Render the video

**Tools:** Hermes skills, brand token profile, Remotion or FFmpeg renderer, FFprobe

# Create a Custom Branded Video Skill

---

## Test it

verify.py must print PASS Lab 05. The custom skill is discoverable, uses relative or Hermes template paths, creates an MP4, passes the technical probe and meets every required brand token.

# Recap — Video Tools, Hermes Skills and Custom Brand Production

1

You can now: LO2: build a governed tool registry and install learner-safe skills with recorded permissions and fallbacks

2

You can now: LO2: package brand, tone and style rules into a reusable skill and verify the rendered output

## TOPIC 03

# Multi-Agent Kanban, YouTube Release and Scheduled Publishing

Agent role contracts | Delegation and parallel work | Durable Kanban | Approval hash and cron release

## Key Concepts — Multi-Agent Kanban, YouTube Release and Scheduled Publishing

1

### Role contract

One agent, one owned deliverable, a bounded tool list and one measurable completion rule.

2

### Delegation context

The exact goal, paths, inputs, constraints and acceptance tests handed to a child agent.

3

### Durable Kanban

Task state and dependencies that survive restarts and deliberate human pauses.

4

### Approval hash

A named decision bound to the immutable hash of the release package it authorises.

# Agent Role Contracts and the Four Specialist Roles

## WHAT

- Each agent receives a contract: a goal, a bounded list of allowed tools, one named deliverable and an explicit completion rule. This course uses four roles. The research agent reads the brief, retrieves sources, scores evidence and hands off claims with retrievable URLs and stated limitations. The video production agent consumes approved claims, creates the shot plan, invokes the brand skill and returns a master with its hash. The independent review agent inspects evidence against a rubric and creates timecoded findings. The upload agent verifies approval, builds metadata and uploads privately.

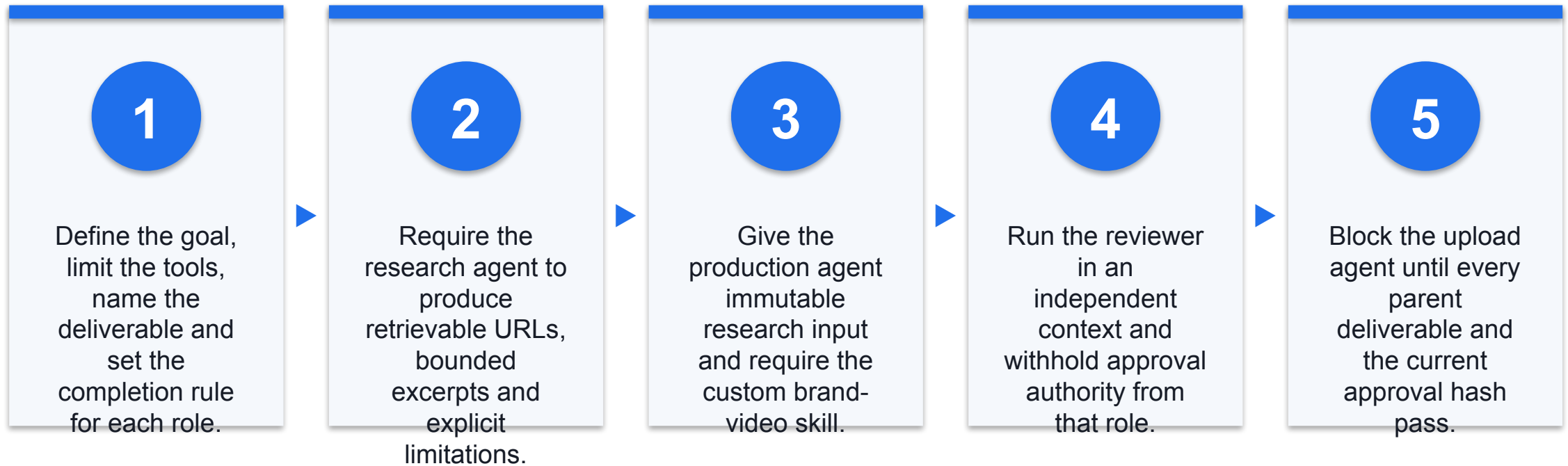
## WHY IT MATTERS

- Roles without contracts drift into each other until no agent owns the outcome and two agents rewrite the same artifact. One deliverable per agent makes ownership unambiguous and makes a stalled pipeline diagnosable — the incomplete deliverable identifies the responsible role. The reviewer is deliberately denied approval authority: a reviewer that can approve its own findings provides no independent check, because the same assumptions that produced the work would clear it.

## HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — Agent Role Contracts and the Four Specialist Roles



## In Practice — Agent Role Contracts and the Four Specialist Roles

### WORKED EXAMPLE

- agent-contracts.yaml validates all four specialist roles against the required fields.
- research-handoff.json contains approved sources and claim identifiers.
- review-handoff.json contains timecoded findings and a QA status but no approval decision.

### DECISION GUIDE

- **Use when**
  - Distinct roles have genuinely different tools, inputs or review criteria.
  - An independent check is needed before a consequential external action.
- **Avoid when**
  - One bounded prompt and one tool would complete the task more reliably.
  - Agents would share unrestricted credentials or be able to overwrite accepted artifacts.

## Quality Lens — Agent Role Contracts and the Four Specialist Roles

### Failure signal

- Two agents disagree about which artifact is current.

### Repair move

- Give each deliverable one owning role and one immutable version identifier.

### Quality evidence

- Every factual claim in the output traces to a sourced claim identifier.

# Delegation Context, Parallel Work and the Durable Kanban

## WHAT

- A delegation context packages the goal, the required context, the exact paths and the acceptance tests before a child agent is dispatched. Independent tasks may run in parallel, but the join happens only on explicit verified artifacts rather than on elapsed time. The Hermes Kanban makes this durable: tasks carry an assignee, a status and parent identifiers, and the board survives restarts and deliberate human pauses, which is what allows a production run to span a break, a review or an overnight wait.

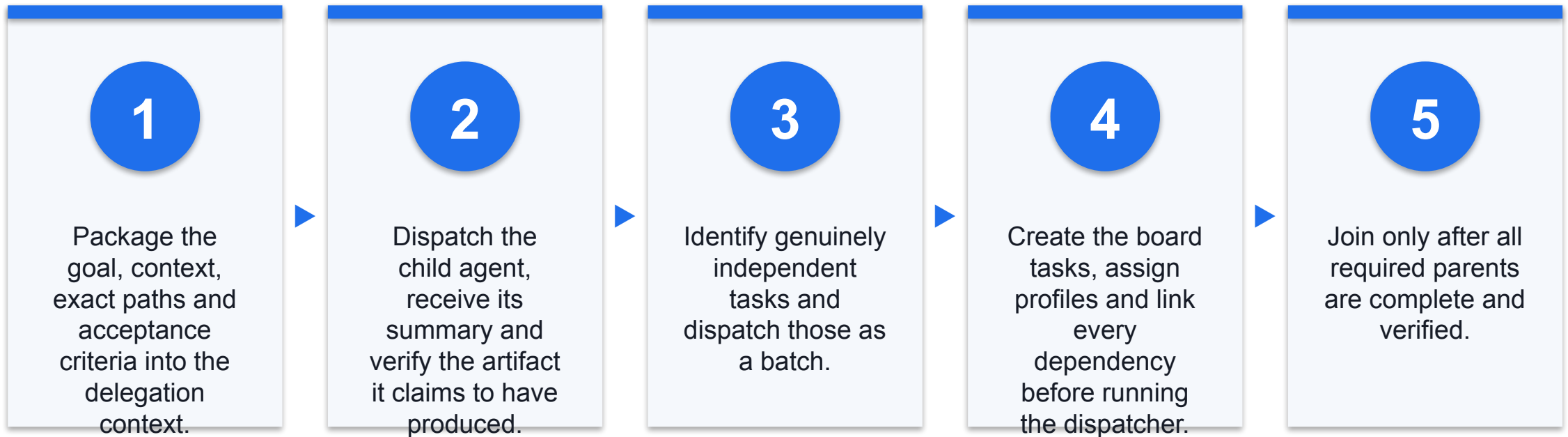
## WHY IT MATTERS

- A child agent that receives a goal without paths, inputs and tests will produce something plausible that does not fit the pipeline, and the mismatch is usually discovered at the join. Parallelising dependent work is worse: it appears faster while silently skipping a dependency. A durable board is what separates a multi-agent workflow from a long single conversation — work that must survive a restart cannot live only in a session, and a paused human review is a normal state rather than a failure.

## HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — Delegation Context, Parallel Work and the Durable Kanban



## In Practice — Delegation Context, Parallel Work and the Durable Kanban

### WORKED EXAMPLE

- delegation-log.json records the prompt, the agent and the verified result.
- join-ledger.json shows every required parent complete before the join proceeded.
- kanban-export.json shows the full dependency chain with no missing dependency.

### DECISION GUIDE

- **Use when**

- Work crosses several agents and must survive restarts or a human pause.
- Independent tasks exist and the join can be defined on verified artifacts.

- **Avoid when**

- The tasks are dependent, so parallel dispatch would skip a dependency.
- The work fits one session and gains nothing from durable state.

## Quality Lens — Delegation Context, Parallel Work and the Durable Kanban

### Failure signal

- A task repeats the same failure and the pipeline retries it indefinitely.

### Repair move

- Cap attempts, classify the failure and block repeated failures for human intervention.

### Quality evidence

- Task history shows checkpoints, the review and the final evidence for each task.

## Human Approval, the YouTube Contract and Publishing Idempotency

### WHAT

- Before release the package is frozen and hashed, and a named reviewer approves that specific hash. Immediately before upload the current master hash is compared against the approved hash; a mismatch blocks the upload. The YouTube videos.insert contract requires OAuth authorisation, a snippet, a status block carrying the privacy setting and, where applicable, the synthetic-media disclosure. Uploads default to private. An idempotency key is created before the attempt and reconciled after it, so one approved package yields at most one committed video identifier.

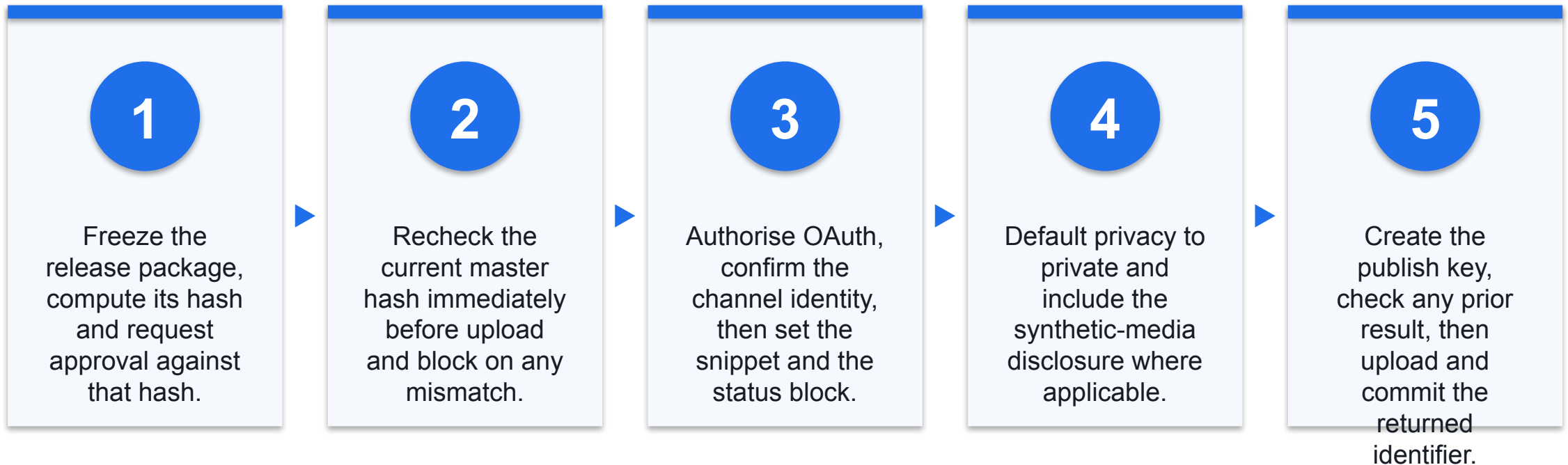
### WHY IT MATTERS

- Publishing is irreversible in a way that almost nothing earlier in the pipeline is. Approving a hash rather than a filename closes the gap where an artifact changes between approval and upload. Defaulting to private means a mistake is recoverable by deleting a private video rather than by retracting a public one. Reconciling an idempotency key before a retry prevents the specific failure where a network timeout on a successful upload produces a second copy on the channel.

### HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — Human Approval, the YouTube Contract and Publishing Idempotency



## In Practice — Human Approval, the YouTube Contract and Publishing Idempotency

### WORKED EXAMPLE

- approval-ledger.json proves the reviewer, the time, the decision and the approved hash.
- request-preview.json and the private upload receipt match the approved package.
- publication-ledger.csv contains exactly one committed video identifier per idempotency key.

### DECISION GUIDE

- **Use when**

- An external, irreversible action is about to be taken on an approved artifact.
- Retries are possible and duplicate posts would be damaging.

- **Avoid when**

- The approved hash does not match the current master hash.
- Channel identity, disclosure or rights are unresolved, or public visibility is proposed without supervision.

## Quality Lens — Human Approval, the YouTube Contract and Publishing Idempotency

### Failure signal

- A retry after a timeout produces a second video on the channel.

### Repair move

- Reconcile the prior idempotency key and its committed video identifier before retrying.

### Quality evidence

- The duplicate publish rate is zero across all recorded attempts.

# Hermes Cron and Controlled Scheduled Release Operations

## WHAT

- A cron job runs in a fresh session, so its prompt must be self-contained: every required path, gate and stop rule is written into the job, and the skills it needs are attached explicitly. The job is created paused. It is exercised with a dry run whose output is inspected, and the next scheduled run is confirmed before any cadence is enabled. Scheduled work in this course prepares the release and performs the private upload; public visibility remains a separate, explicitly approved decision.

## WHY IT MATTERS

- A scheduled job inherits nothing from the session that created it, so a prompt that relies on conversational context will behave differently at three in the morning than it did during testing. Creating the job paused makes that difference observable before it matters. Restricting the schedule to preparation and private upload means the worst outcome of a scheduling defect is an unwanted private draft rather than an unapproved public post.

## HUMAN CHECK

- Ground the task in approved facts.
- Review evidence, privacy, rights and tone.
- Approve, edit or stop before use.

## How It Works — Hermes Cron and Controlled Scheduled Release Operations



## In Practice — Hermes Cron and Controlled Scheduled Release Operations

### WORKED EXAMPLE

- cron-preview.json records the schedule, the next run and the paused state.
- operations-ledger.csv records each run, its outcome and the owner's decision.
- The dry run cannot publish an unapproved or duplicate video.

### DECISION GUIDE

- **Use when**
  - A release cadence is required and the preparation steps are already reliable.
  - The job can be made fully self-contained and exercised with a dry run first.
- **Avoid when**
  - The job prompt still depends on context from the session that created it.
  - Enabling the cadence would allow a public post without a separate approval.

## Quality Lens — Hermes Cron and Controlled Scheduled Release Operations

### Failure signal

- A job that passed interactively behaves differently on its first scheduled run.

### Repair move

- Move every implicit path, gate and stop rule into the job prompt and re-run the dry run.

### Quality evidence

- The on-time success rate holds with zero unauthorised posts.

## Hands-On Labs — Multi-Agent Kanban, YouTube Release and Scheduled Publishing

### Lab 6

- Build the Multi-Agent Video Workflow

### Lab 7

- Orchestrate Kanban Review and YouTube Upload

### Lab 8

- Schedule Controlled Video Publishing with Hermes Cron

# Build the Multi-Agent Video Workflow

## LAB 6

You will define and simulate four isolated Hermes roles for research, video creation, independent review and approved YouTube upload. Each role receives a bounded tool list and a named deliverable, the reviewer runs in an independent context without approval authority, and the uploader stays blocked until all parent evidence and the current approval hash pass.

### You'll build

**A multi-agent-plan.json with four verified handoff records.**

1 Define the role contracts

2 Package the context

3 Delegate research

4 Delegate production

**Tools:** Hermes delegation, agent contracts, handoff schema

# Build the Multi-Agent Video Workflow

---

## Test it

verify.py must print PASS Lab 06. All roles have bounded tools and outputs, the reviewer is independent, and upload is blocked until all parent evidence and the current approval hash pass.

# Orchestrate Kanban Review and YouTube Upload

## LAB 7

You will create a Hermes Kanban dependency chain, assign profiles, link every parent, require review, approve the exact package hash and then prepare or execute a private YouTube upload under explicit human authorisation with an idempotency key.

### You'll build

**A kanban-export.json, approval-ledger.json and a private-upload receipt or dry-run preview.**

1 Create the board tasks

2 Assign the profiles

3 Link the dependencies

4 Request review

**Tools:** Hermes Kanban, approval ledger, YouTube Data API videos.insert

## Orchestrate Kanban Review and YouTube Upload

---

### Test it

verify.py must print PASS Lab 07. The Kanban graph has no missing dependency, review is required, the request points to the approved master, privacy is private, and no credential is present in any artifact.

# Schedule Controlled Video Publishing with Hermes Cron

## LAB 8

You will write a self-contained cron prompt containing every required path, gate and stop rule, attach the custom video skill explicitly, create the schedule paused with an explicit timezone, trigger a dry run, inspect the next run time and approve enablement only after reviewing the evidence.

### You'll build

**A cron-preview.json with an operations-ledger.csv.**

1 Write the self-contained job

2 Attach the video skill

3 Create the paused cron

4 Trigger the dry run

**Tools:** Hermes cron, attached skills, operations ledger

## Schedule Controlled Video Publishing with Hermes Cron

---

### Test it

verify.py must print PASS Lab 08. The schedule and timezone are explicit, the prompt is self-contained, the custom skill is attached, the job begins paused, and the dry run cannot publish an unapproved or duplicate video.

# Recap — Multi-Agent Kanban, YouTube Release and Scheduled Publishing

1

You can now: LO3: define four bounded specialist roles and prove that upload stays blocked until every gate passes

2

You can now: LO3: run a durable dependency chain and bind a named approval to the exact release hash

3

You can now: LO3: create a paused, self-contained scheduled job and enable a cadence only after evidence review



WRAP-UP

# Course Summary & Next Steps

# What You Achieved

1

## Plan the Video

Develop an editing strategy and work plan in Hermes Agent using MiniMax M3, structured video prompts and governed tool selection.

2

## Build the Video

Create and customise an evidence-backed video with Hermes skills, Remotion, Manim, Higgsfield-compatible requests and deterministic media checks.

3

## Orchestrate and Release

Orchestrate research, production, review and YouTube release agents through a durable Kanban board and controlled scheduled publishing.

# Continue Building Safely

1

Replace one deterministic fallback at a time and retain its contract and evidence.

2

Create test jobs for a missing input, a malformed shot plan, a tool failure, and a denied approval.

3

Keep uploads private until channel permissions and an accountable release owner are confirmed.

4

Measure accepted masters, repair rate, cost, and latency before enabling any cadence.

# Keep Practising After the Course

1

Re-run the labs on your own machine — repetition is what makes the skills stick.

2

Apply the techniques to a real process or project in your own organisation.

3

Keep your downloaded C436 course files and use the lab checkpoints to restart safely.

4

Explore the follow-on courses at [www.tertiarycourses.com.sg](http://www.tertiarycourses.com.sg).

C436 | AGENTIC AI FOR VIDEO CREATION

# Thank You!

You can now configure Hermes Agent, engineer video prompts, build a custom branded video skill, and operate a reviewed, evidence-led release pipeline.